Neural Networks

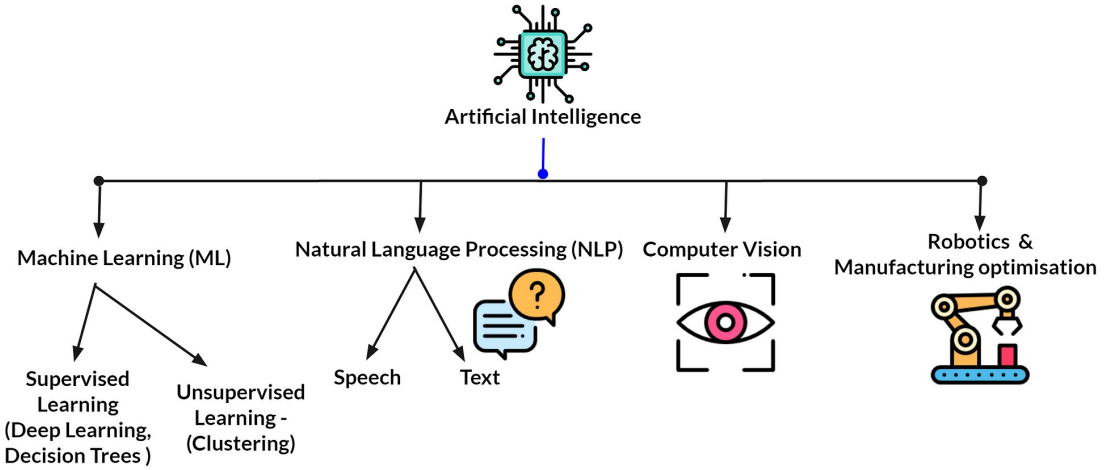
Dr. Satadisha Saha Bhowmick

Preceptor

THE UNIVERSITY OF CHICAGO
DATA SCIENCE INSTITUTE

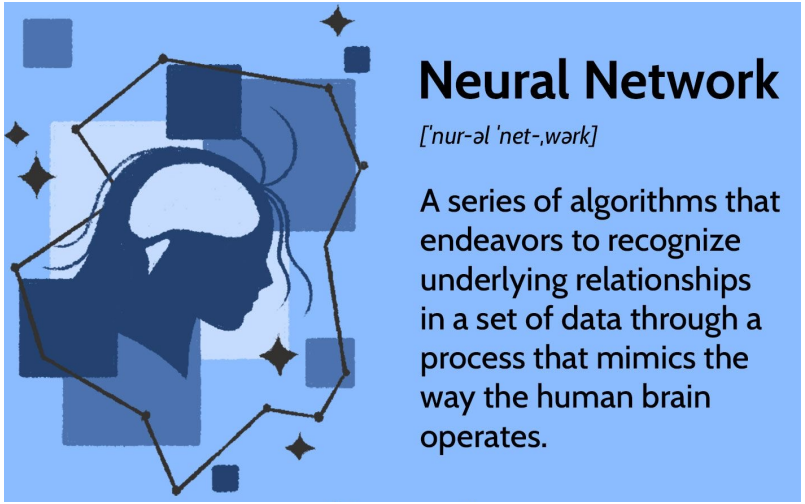
Objectives

- Motivation
- Perceptron
- Sigmoid Neuron
- Multi Layer Perceptron
- Activation Functions
- Training a Neural Network



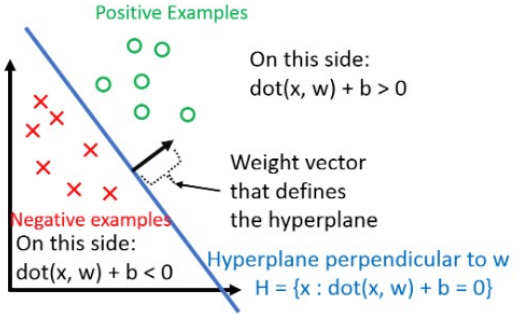
Motivation

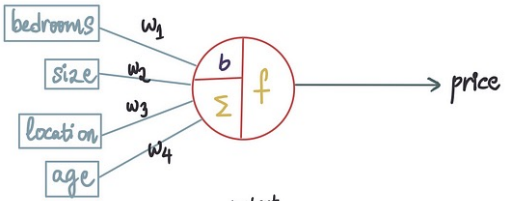
- Neural Networks are interconnected systems of computational nodes modelled after the human cognitive system.
- Can represent models drawn from complex function spaces that are not limited by linearity.
- Can automatically recognize hidden patterns or correlations in data.
- Flexible framework that works for both regression and classification problems.
- Suitable for building complex AI systems
    - Natural Language Processing
    - Image Processing
    - Speech Recognition
    - Targeted marketing
    - Financial predictions
    - Robotics control systems
- Advanced computational technologies (GPUs) make implementation easy and scalable for big data problems

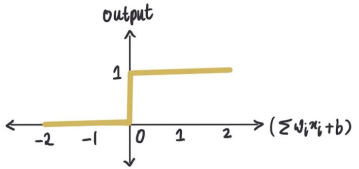


Perceptron

- Simplest type of Neural Network is a single artificial neuron or Perceptron
- Developed by Frank Rosenblatt in 1957 at Cornell University
  - The 'first' machine learning algorithm!
- Assumptions:
  - Binary Classification (i.e., $y_i \in \{0,1\}$)
  - Data is linearly separable
- 3 components: 1) Weight, 2) Bias, and 3) Activation Function
- Classifier: $h(x) = w^T x + b$
  - Output $y = \begin{cases} 0 \; if \; h(x) \leq 0 \\ 1 \; if \; h(x) > 0 \end{cases}$
  - Linear Combination: Matrix notation - b is the bias term and w is the weight
  - Activation Function: Binary Step Function. Piecewise linear instead of non-linear
- Perceptron are simple and computationally universal but limited in decision making





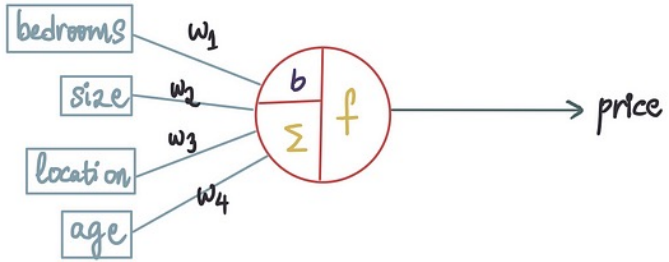


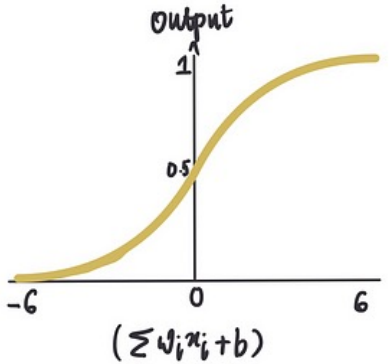
Sigmoid Neuron

- Network of perceptrons can be effective but small changes can activate or deactivate a lot of neurons.
  - Can result in uncontrolled change across the entire network
  - Sigmoid Neurons ensure small changes in weights and bias in a neuron can only cause small changes in output
- Remember Logistic Regression? That's a single sigmoid neuron.
  - One of the simplest possible neural networks.

$$z = w_1 x_1 + w_2 x_2 + \cdots . + w_m x_m + b = \sum_{i=1}^{m} w_i x_i + b$$

$$f(z) = \frac{1}{1 + e^{-z}}$$





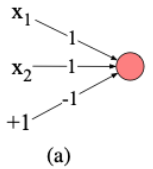
The XOR problem

- A single neuron cannot compute some very basic logical functions of its input. Consider three popular ones: AND, OR, and, XOR.
- Consider the perceptron model that we just learn which is essentially a single neuron

$$y = \begin{cases} 0, if\ wT \cdot x + b \leq 0 \\ 1, if\ wT \cdot x + b > 0 \end{cases}$$

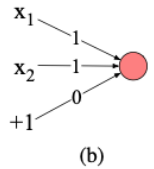
- Can build a perceptron for the logical AND and OR gates of binary inputs
  - Logical AND: $w_1 = 1, w_2 = 1, b = -1$
  - Logical OR: $w_1 = 1, w_2 = 1, b = 1$
  - These are just one of the possible combinations of weights and biases
- Not possible to build a single perceptron to compute logical XOR
  - For two-dimensional input x_1 and x_2 the perceptron equation looks like: $w_1 x_1 + w_2 x_2 + b = 0$. This line acts as the decision boundary with output 0 assigned to all points on one side of the line and 1 assigned to all points on the other side.
  - We can see for all the possible logical inputs (00, 01, 10, 11) and a line drawn with one set of parameters for an AND and OR classifier. For the XOR case, we do not have a way to draw a line that separates the positive cases of XOR (01 and 10) from the rest (00 and 11). XOR is not a **linearly separable** function.

AND			OR			XOR		
x1	x2	y	x1	x2	y	x1	x2	y
0	0	0	0	0	0	0	0	0
0	1	0	0	1	1	0	1	1
1	0	0	1	0	1	1	0	1
1	1	1	1	1	1	1	1	0



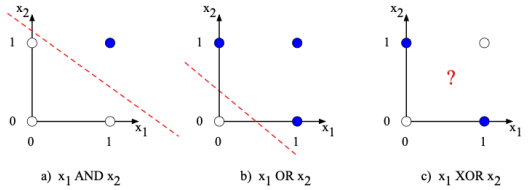
(a)

Logical AND



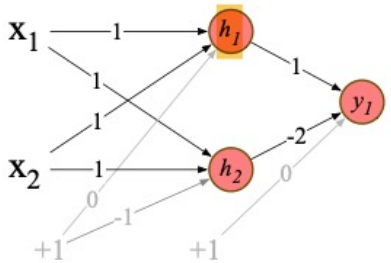
(b)

Logical OR



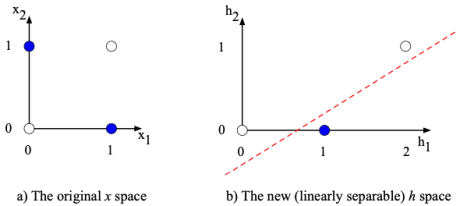
XOR Problem Solution

- XOR function cannot be calculated by a single perceptron, but it can be calculated by a layered network of perceptron units.
- Network Details:
  - Input Layer: 2-dimensional binary input
  - Hidden Layer: 2 layers of ReLU (activation function) neurons
  - Output Layer: One ReLU neuron



Let's walk through what happens with the input x = [0, 0]. If we multiply each input value by the appropriate weight, sum, and then add the bias *b*, we get the vector [0, -1], and we then apply the rectified linear transformation to give the output of the h layer as [0, 0]. Now we once again multiply by the weights, sum, and add the bias (0 in this case) resulting in the value 0.

It's also instructive to look at intermediate results, the outputs of the two hidden nodes *h*1 and *h*2. For the input [0,0] the h vector is [0,0]. Also, hidden representations of the two input points x = [0, 1] and x = [1, 0] (cases with XOR output = 1) are merged to the single point h = [1, 0]. The merger makes it easy to linearly separate the positive and negative cases of XOR. In other words, we can view the hidden layer of the network as forming a representation of the input.



a) The original *x* space b) The new (linearly separable) *h* space

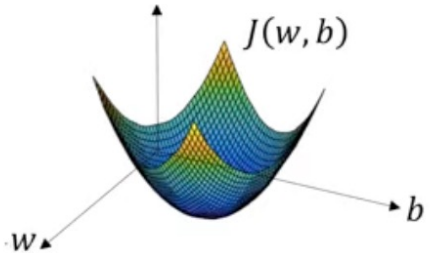
Your Turn: Work through the computation for the remaining inputs of [0,1], [1,0], and [1,1].

Notations

- Conventionally we use matrix notations to express the computations that take place within neural networks.
- A single datapoint (x,y) where $x \in \mathbb{R}^{n_x}$, $y\{0,1\}$
- m_{train}: m training examples:$\{ \left(x^{(1)}, y^{(1)}\right), \left(x^{(2)}, y^{(2)}\right), \dots, (x^{(m)}, y^{(m)})\}$
- $X = \begin{bmatrix} | & | & | & & | \\ x^{(1)} & x^{(2)} & x^{(3)} & \dots & x^{(m)} \\ | & | & | & & | \end{bmatrix}$
- Y= $[y^{(1)}, y^{(2)}, \dots, y^{(m)}]$
- X shape (n_x,m) and Y shape (1,m)
- Given x, predicts $\hat{y} = P(y = 1|x) = \sigma(w^T x + b)$
- Parameters: $w \in \mathbb{R}^{n_x}$, $b \in \mathbb{R}$
- Loss Function: $\mathfrak{L}(\hat{y}, y) = -(y \log \hat{y} + (1 - y)\log(1 - \hat{y}))$
- Cost Function: $\mathcal{J}(w, b) = \frac{1}{m}\sum_{i=1}^{m} \mathfrak{L}(\hat{y}^{(i)}, y^{(i)}) = -\frac{1}{m}\sum_{i=1}^{m}[y^{(i)} \log \hat{y}^{(i)} + (1 - y^{(i)})\log(1 - \hat{y}^{(i)})]$

Gradient Descent

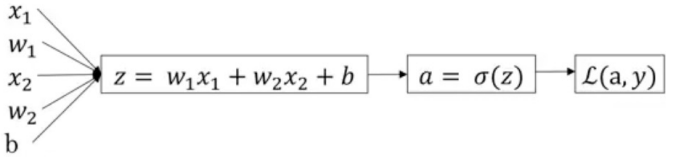
- Optimization is the task of minimizing the cost function incurred by a model parameterized by the model's parameters.
- Gradient Descent is an optimization algorithm used to minimize objective functions/cost functions to find optimal parameter estimates.
- Conditions: Loss function should be convex, continuous and differentiable
- **Algorithm**
  - Initialize w randomly, α is the learning rate hyperparameter
  - Repeat until convergence
    - $w^{t+1} \coloneqq w^t - \alpha \frac{d\mathcal{J}(w)}{dw}$
  - If $\|w^{t+1} - w^t\|_2 < \epsilon$, convergence is reached
- Learning rate controls how big a step we take during one iteration of Gradient Descent.
- No matter where parameters are initialized, Gradient Descent always moves towards the global minima
- For logistic regression we want to find parameters *w, b* such that $\mathcal{J}(w, b)$ is minimized. Hence, we take partial derivatives for updates.
  - $w \coloneqq w - \alpha \frac{\partial \mathcal{J}(w,b)}{\partial w}$
  - $b \coloneqq b - \alpha \frac{\partial \mathcal{J}(w,b)}{\partial b}$
  - In shorthand, $dw = \frac{\partial \mathcal{J}(w,b)}{\partial w}$ and $\mathrm{db} = \frac{\partial \mathcal{J}(w,b)}{\partial w}$



Logistic Regression Optimization

Consider a simple Logistic regression model with m examples each having two features.
For m examples, *a single iteration* of Gradient Descent consists of the following computations

- $\mathcal{J} = 0;\ dw_1 = 0; dw_2 = 0; db = 0$
- `for i=1 to m`
  - **Forward Pass**
$$\begin{cases} z^{(i)} = w^T x^{(i)} + b \\ a(i) = \sigma(z^{(i)}) \\ \mathcal{J} \mathrel{+}= -[y^{(i)} \log a^{(i)} + (1 - y^{(i)}) \log(1 - a^{(i)})] \end{cases}$$
  - **Backward Pass**
$$\begin{cases} dz^{(i)} = a^{(i)} - y^{(i)} \\ dw_1 \mathrel{+}= x_1^{(i)} dz^{(i)} \\ dw_2 \mathrel{+}= x_2^{(i)} dz^{(i)} \\ db \mathrel{+}= dz^{(i)} \end{cases}$$
- $\mathcal{J} /= m;\ dw_1$`/=m;` dw_2`/=m;` db`/=m;`
- `Updates: learning rate` α `is a hyperparameter`
  - $w_1 := w_1 - \alpha dw_1$
  - $w_2 := w_2 - \alpha dw_2$
  - $b := b - \alpha db$



Partial Derivatives are computed using the **chain rule**!

In practice we repeat Gradient Descent over multiple iterations.
Nested loop implementation is computationally inefficient.
Use **vectorization**!

Vectorizing Logistic Regression

Vectorization empowered with GPUs is the cornerstone of Neural Network computations. *Get comfortable with matrices!*
Vectorized versions (over m examples) are denoted in capital. Iterating over m examples reduced to a single matrix computation.

`for iter in range(1000)`

Forward Pass/Forward Propagation

- **for m examples**
  - $z^{(1)} = w^T x^{(1)} + b;\ a^{(1)} = \sigma(z^{(1)})$
  - $z^{(2)} = w^T x^{(2)} + b;\ a^{(2)} = \sigma(z^{(2)})$ … and so on!

$$X = \begin{bmatrix} | & | & | & & | \\ x^{(1)} & x^{(2)} & x^{(3)} & \dots & x^{(m)} \\ | & | & | & & | \end{bmatrix} \quad w = \begin{bmatrix} w_1 \\ \dots \\ w_{n_x} \end{bmatrix}$$

$$Z = w^T X + b = \left[z^{(1)}, z^{(2)}, \dots, z^{(m)}\right]$$

$$A = \left[\sigma\left(z^{(1)}\right), \sigma\left(z^{(2)}\right), \dots, \sigma\left(z^{(m)}\right)\right] = \left[a^{(1)}, a^{(2)}, \dots, a^{(m)}\right]$$

Notice all the dimensions of the matrices involved

- X: $n_x \times m$ w: $n_x \times 1$ b: real-valued (1×1), broadcasted
- Z:$1\times m$ A: $1\times m$

Backward Pass/Backpropagation

Note that partials have the same dimensions as the original matrices

- $dz^{(1)} = a^{(1)} - y^{(1)}; dz^{(2)} = a^{(2)} - y^{(2)}$
- With $A = \left[a^{(1)}, a^{(2)}, \dots, a^{(m)}\right]$ and Y = $\left[y^{(1)}, y^{(2)}, \dots, y^{(m)}\right]$
- $dZ = \left[dz^{(1)}, dz^{(2)}, \dots, dz^{(m)}\right] = A - Y$
  $= [a^{(1)} - y^{(1)}, a^{(2)} - y^{(2)}, \dots, a^{(m)} - y^{(m)}]$
- $db = \frac{1}{m}\sum_{i=1}^{m} dz^{(i)} = \frac{1}{m} np.sum(dZ)$
- $dw = \frac{1}{m} X dZ^T = \frac{1}{m} \begin{bmatrix} | & | & | & & | \\ x^{(1)} & x^{(2)} & x^{(3)} & \dots & x^{(m)} \\ | & | & | & & | \end{bmatrix} \begin{bmatrix} dz^{(1)} \\ . \\ dz^{(m)} \end{bmatrix}$

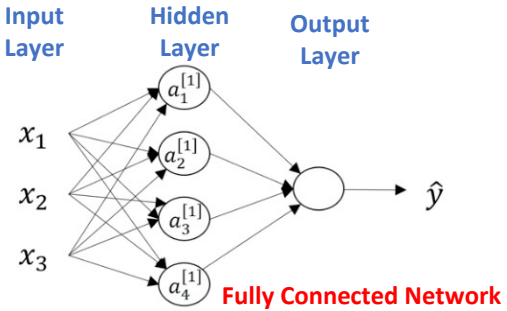
Neural Networks

- So far, we have seen Single Layer Perceptron/Networks
  - Input and Output Layers; no '**Hidden**' Layers
- Deep Neural Networks
  - One or more **Hidden** layers between input and output layers
  - Each node in the hidden/output layer consists of: 1) Weight, 2) Bias, and 3) Activation Function
- All calculations are conducted by vectorizing the inputs, weights, biases and hidden layer outputs into a single matrix or tensor (for higher dimensional operations).

$$\boldsymbol{z^{[1]}} = \begin{bmatrix} -w_1^{[1]T} - \\ -w_2^{[1]T} - \\ -w_3^{[1]T} - \\ -w_4^{[1]T} - \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} + \begin{bmatrix} b_1^{[1]} \\ b_2^{[1]} \\ b_3^{[1]} \\ b_4^{[1]} \end{bmatrix} = W^{[1]}x + b^{[1]}\,;\, \boldsymbol{a^{[1]}} = \begin{bmatrix} a_1^{[1]} \\ a_2^{[1]} \\ a_3^{[1]} \\ a_4^{[1]} \end{bmatrix} = \sigma\left(z^{[1]}\right)$$

$$\boldsymbol{z^{[2]}} = W^{[2]}a^{[1]} + b^{[2]};\, \boldsymbol{a^{[2]}} = \sigma\left(z^{[2]}\right)$$

$$X = \begin{bmatrix} | & | & | & & | \\ x^{(1)} & x^{(2)} & x^{(3)} & \dots & x^{(m)} \\ | & | & | & & | \end{bmatrix};\, A^{[1]} = \begin{bmatrix} | & | & | & & | \\ a^{[1](1)} & a^{[1](2)} & a^{[1](3)} & \dots & a^{[1](m)} \\ | & | & | & & | \end{bmatrix}$$



$$z_1^{[1]} = w_1^{[1]T}x + b_1^{[1]},\; a_1^{[1]} = \sigma(z_1^{[1]})$$
$$z_2^{[1]} = w_2^{[1]T}x + b_2^{[1]},\; a_2^{[1]} = \sigma(z_2^{[1]})$$
$$z_3^{[1]} = w_3^{[1]T}x + b_3^{[1]},\; a_3^{[1]} = \sigma(z_3^{[1]})$$
$$z_4^{[1]} = w_4^{[1]T}x + b_4^{[1]},\; a_4^{[1]} = \sigma(z_4^{[1]})$$

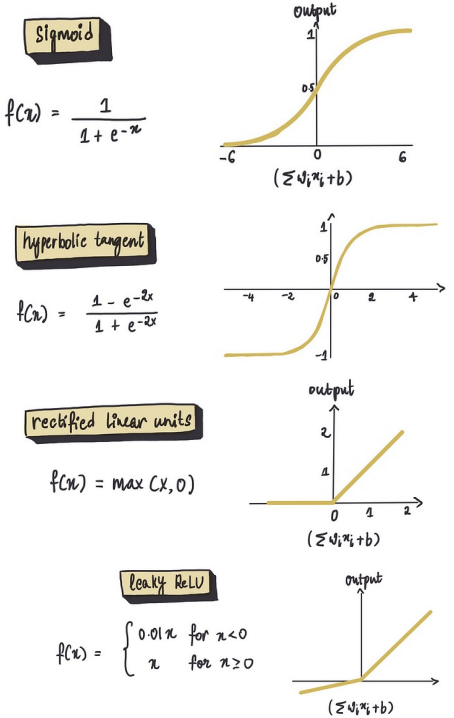
$$\boldsymbol{Z^{[1]} = W^{[1]}X + b^{[1]};\, A^{[1]} = \sigma(Z^{[1]});}$$
$$\boldsymbol{Z^{[2]} = W^{[2]}A^{[1]} + b^{[2]};\, A^{[2]} = \sigma(Z^{[2]});}$$

Activation Functions

- Nonlinearity is an essential component of a neural network!
  - Injected into the network using a non-linear activation function.
- Why non-linear activation function?
  - If you replace the activation function in the previous slide with an identity function what you have is plain old linear regression!

If, $a^{[1]} = z^{[1]} = W^{[1]}x + b^{[1]}$ then,

$$a^{[2]} = W^{[2]}\left(W^{[1]}x + b^{[1]}\right) + b^{[2]} = \underbrace{\left(W^{[2]}W^{[1]}\right)}x + \underbrace{\left(W^{[2]}b^{[1]} + b^{[2]}\right)} = W'x + b'$$

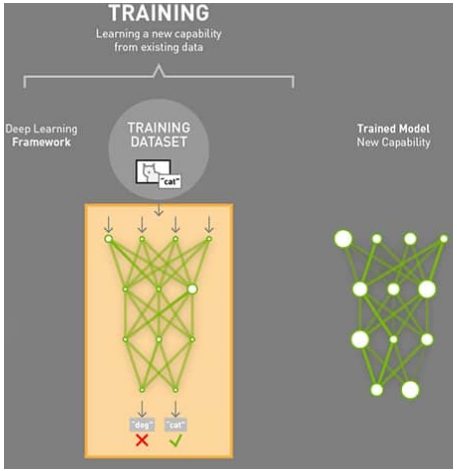


- **Sigmoid** : Good for binary classification or probabilities. Range (0,1)
- **Tan-h** : Hyperbolic Tangent function, similar to sigmoid but centered around 0. Range (-1,1)
- **ReLU** : Rectified Linear Units, one of the most simple but effective choices. Unlike sigmoid or tan-h this function has a defined gradient at positive values that is useful during Gradient Descent (doesn't slow down learning). Gradient disappears for non-positive values.
- **Leaky ReLU** : While ReLU sets all negative inputs to zero, Leaky ReLU allows a small, non-zero, constant output for negative inputs. This function always has a non-zero gradient.

For internal nodes we typically avoid Sigmoid activation function but good for output nodes.

Cost Functions

- Once we have the architecture for the network, we need to have a metric for comparison of two networks: "how good" a neural network did with respect to the given training sample and the expected output. The difference between the expected and predicted output is called a ‘loss’
- How to count your losses given the observed value y and predicted y estimate $\hat{y}$?
  - Squared Error – $(y - \hat{y})^2$
  - Absolute Error – $|y - \hat{y}|$
  - Binary Cross Entropy Loss – $|y \log \hat{y} + (1 - y) \log(1 - \hat{y})|$; y is binary and $\hat{y}$ is between 0 and 1
  - Cross Entropy Loss – $\sum_{i=1}^{c} y_i \log \hat{y}_i$; used for multi-class classification over c classes. The output in this case is a probability distribution over c classes where $\hat{y}_i$ is the probability estimate of the i-th class.
- Cost Function: An aggregate of these losses over all training examples
- $\hat{y}$ is a function of all the features and network parameters. We have the former as inputs and seek to ‘optimize’ the latter.



Training Neural Networks

- Network Learning involves using an optimization algorithm to find the weights and biases that minimize a cost function.
  - Find partial derivatives of the cost function w.r.t. the network parameter (weights and biases) we want to optimize.
    - **Backpropagation** – An algorithm to compute the gradient vector efficiently. Recursive application of the '**chain rule**' to update parameters by accumulating upstream gradient at every step of the compounded neural network function.
  - Solve for the optimal parameter value by equating the partial derivative of the cost function w.r.t. the parameter to zero.
    - Might not have closed form solution! Use iterative algorithms like Gradient Descent.
    - Conduct forward propagation through the network all the way to the loss function.
    - Use backpropagation to obtain partial derivative and adjust value in the gradient descent step until convergence.
- **Optimizing Gradient Descent**: Regular Gradient Descent involves computing the full cost function – loss function for all training examples – to adjust a parameter value in a single iterative step.
  Computationally expensive for large datasets!
  - Mini Batch Gradient Descent: Randomly split training step into **mini-batches**, compute a gradient descent step according to a mini-batch. All the steps that are required to allow the entire training set to influence parameter updates collectively form an '**epoch**'.
  - Stochastic Gradient Descent: When mini-batch size is exactly one example. Every step of gradient descent is taken on one example. Although S.G.D. is computationally efficient and fast, updates based on individual examples can also cause a lot of random updates (hence stochastic)! It also never fully converges but hovers asymptotically close to the local minima.